

JSyn

Quick Dive (1 hour)

www.softsynth.com/jsyn

What is JSyn

- Modular synthesis toolkit for Java
- >100 Unit generators: oscillators, filters, etc.
- Connect and control units from Java
- Runs on Mac, Windows, Linux
- Android port in progress (2014)
- Swing Tools: rotary knob, scope, envelope

JSyn Benchmarks on Android

- Subtractive Synth voice with SawBL, LowPass, AmpEnv, FilterEnv takes 2.2% of realtime on Molly.
- Comparative benchmarks (seconds)

Benchmark	Molly	Nexus 5 Dalvik	Nexus 5 ART
FFT Double	2.665	3.222	2.496
FFT Single	1.955	2.567	2.073
Subtractive	10.905	17.981	8.404

Starting JSyn

```
// Use JSyn factory class to create a Synthesizer.
```

```
Synthesizer synth = JSyn.createSynthesizer();
```

```
// Start default stereo output at 44100 Hz.
```

```
synth.start();
```

```
// OR use a specific device
```

```
synth.start( 44100, inputDeviceID, numInputChannels,  
            outputDeviceID, numOutputChannels );
```

Add and Connect Unit Generators

```
// Add a tone generator.
```

```
synth.add( osc = new SineOscillator() );
```

```
// Add a stereo audio output unit.
```

```
synth.add( lineOut = new LineOut() );
```

```
// Connect the oscillator to both channels of the output.
```

```
osc.output.connect( 0, lineOut.input, 0 );
```

```
osc.output.connect( 0, lineOut.input, 1 );
```

Set Port Values

```
// Set the frequency and amplitude for the sine wave.  
osc.frequency.set( 345.0 ); // in Hertz  
osc.amplitude.set( 0.6 ); // 0.0 to 1.0 for audio
```

```
// We only need to start the LineOut.  
// It will pull data from the oscillator.  
lineOut.start();
```

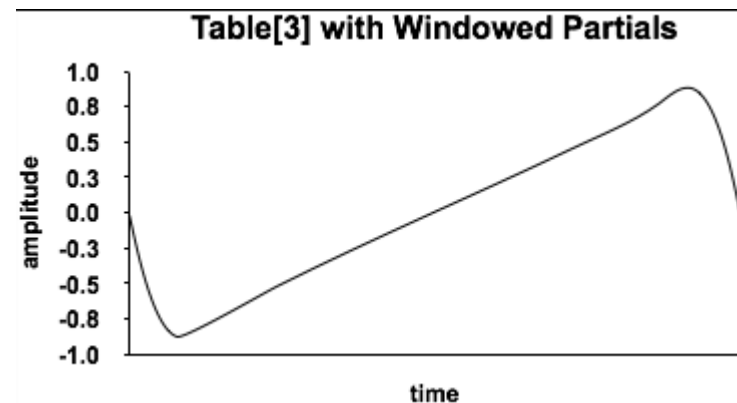
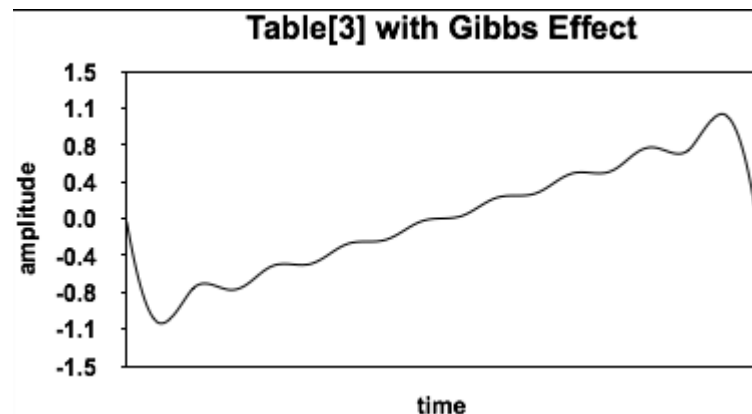
- JSyn now playing a tone!
- Example: **PlayTone.java**

Common Sound Sources

- SineOscillator – pure tone
- SineOscillatorPM – phase modulated, “FM”
- SawtoothOscillatorBL – violin-like sound
- SquareOscillatorBL – reed-like sound
- PulseOscillatorBL – variable width
- WhiteNoise – intense noise
- RedNoise – variable frequency noise
- Example: [SeeOscillators.java](#)

Band-Limited Oscillators

- Sawtooth tables with increasing numbers of partials. Bilinear interpolation.
- Square, Pulse, Impulse made from Sawtooth
- Window partials using raised-cosine to reduce Gibbs effect.



Segmented Envelopes

- Queue blocks of {duration, value} pairs.
- Variable rate player.
- Polymorphic. Use same player for samples.
- Example: `EditEnvelope1`
- Crossfade between overlapping samples.
- Example: `PlaySampleCrossfade`
- Also a DLS2 style DAHDSR envelope.

Unit Generator Ports

```
public UnitInputPort input;  
public UnitOutputPort output;  
  
/* Define Unit Ports used by connect() and set(). */  
public UnitFilter()  
{  
    addPort( input = new UnitInputPort( "Input" ) );  
    addPort( output = new UnitOutputPort( "Output" ) );  
}
```

Unit Generator Generate()

```
public void generate( int start, int limit )
{
    // Get signal arrays from ports.
    double[] inputs = input.getValues();
    double[] outputs = output.getValues();
    for( int i = start; i < limit; i++ ) {
        double x = inputs[i];
        outputs[i] = x * x * x;
    }
}
```

Ramps for Smoothing

- Ramps are Filters with input and output
- LinearRamp – straight line
 - time port in seconds to reach input
- ExponentialRamp – curve, time port
- AsymptoticRamp – never reaches input
 - halfLife port, time to reach half way to input
- ContinuousRamp – S curve
 - model hand on fader

Math and Logic

- Add, mixes, can also use port inputs
- Multiply, gain
- Latch, Schmidt Trigger
- EdgeDetector – Sample and Hold
- Pan
- Integrate – sums input

Other Important Units

- PitchDetector – track guitar
- Delay
- InterpolatingDelay – flange effects
- PowerOf2 – scale LFO for zero pitch offset
- FFT/IFFT (beta)

Granular Synthesis

- GrainFarm schedules N Grains
- Default GrainSource is a sine wave.
- Default GrainEnvelope is RaisedCosine
- SampleGrainFarm uses SampleGrainSource
- Timing based on grain's *duration* and *gap*.
- Example: **PlayGrains.java**

Waveshaping

- Table Lookup or Function Evaluation

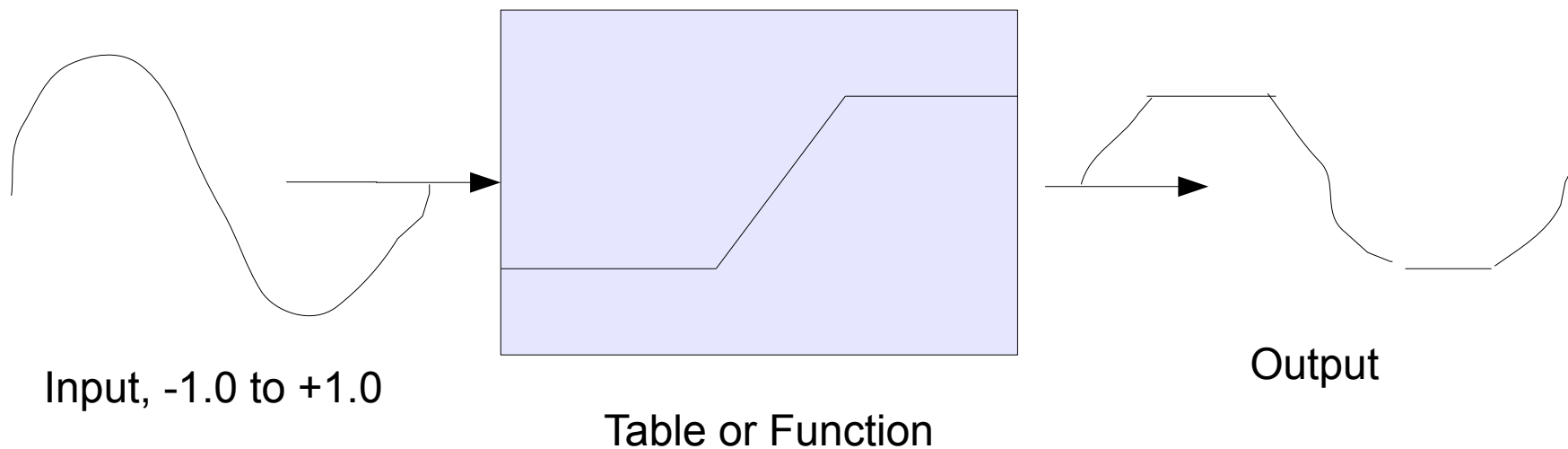


Table Lookup

- `// Define a double precision table`
`double[] data = new double[1024 + 1];`
- `// Create a table lookup object`
`table = new DoubleTable( data );`
`synth.add(looker = new FunctionEvaluator());`
`looker.function.set(looker);`
- Chebyshev Polynomials
- WaveShapingVoice
- Example: **ChebyshevSong**

Syntona Patch Editor

- Pure Java patch editor
- Exports Java source code
- Some composition and performance tools

